

HACKATHON FIAP . GRUPO 32

Plataforma Digital de Atendimento SUS

Relatório Técnico do Projeto

Arquitetura . Segurança . Dados . Mensageria . Infraestrutura . Testes

Equipe: Luis . Thiago . Juliana . Gilmar . Danilo

1. Resumo Executivo

A Plataforma Digital de Atendimento SUS é uma API REST que centraliza o ciclo completo de atendimento ao paciente do Sistema Único de Saúde. O sistema conecta pacientes, médicos, atendentes e administradores em um único ambiente digital, com controle de acesso por perfil e rastreabilidade completa de cada etapa do atendimento.

O MVP entrega um backend funcional e demonstrável com autenticação JWT stateless, agendamento de consultas e exames, registro de resultados, emissão de pareceres, receitas e documentos médicos, além de notificações desacopladas via Apache Kafka.

A solução está implementada como um monólito modular com Spring Modulith, onde cada módulo de negócio segue Clean Architecture própria, com fronteiras verificadas em tempo de build pelo ArchUnit. O sistema conta com 422 testes automatizados passando e cobertura de 80%+.

2. Problema Identificado

O Sistema Único de Saúde (SUS) enfrenta desafios críticos relacionados à fragmentação de informações e à ausência de integração digital entre as etapas do atendimento:

- Informações clínicas dispersas em diferentes unidades de saúde, sem acesso unificado.
- Processos manuais de agendamento, gerando filas, remarcações e cancelamentos sem rastreamento.
- Ausência de histórico centralizado: o paciente não consegue acompanhar sua jornada de forma contínua.
- Resultados de exames e laudos não disponibilizados de forma estruturada e segura.
- Falta de comunicação automática sobre eventos de atendimento (agendamentos, resultados, receitas).
- Dificuldade de auditoria e rastreabilidade das ações clínicas e administrativas.

Princípio central da solução

Separar o dado técnico produzido pelo exame da interpretação clínica do médico. O resultado do exame é persistido independentemente do parecer médico, preservando autoria, histórico, rastreabilidade e permitindo evolução futura do sistema.

3. Descrição da Solução

A plataforma centraliza etapas do atendimento ao paciente do SUS em um único ambiente digital. O sistema começa pelo cadastro dos usuários e permite que pacientes acompanhem seu atendimento, realizem agendamentos, consultem resultados, pareceres, receitas e documentos. Médicos podem analisar resultados remotamente e emitir toda a documentação clínica.

3.1 Objetivos

- Centralizar dados e histórico de atendimento do paciente.

- Facilitar agendamento, remarcação e cancelamento de consultas e exames.
- Disponibilizar resultados de exames de imagem e laboratoriais de forma estruturada.
- Permitir análise remota e emissão de parecer por médicos autorizados.
- Possibilitar emissão e renovação de receitas e documentos médicos.
- Notificar usuários sobre eventos relevantes do atendimento via eventos de domínio.
- Aplicar controle de acesso por perfil com Spring Security e JWT.

3.2 Módulos do MVP

Módulo	Escopo
Usuários e segurança	Cadastro de usuários, autenticação JWT, associação de roles, controle de acesso por perfil.
Consultas	Agendamento, remarcação, cancelamento e acompanhamento de consultas.
Exames	Solicitação médica, agendamento, registro da realização e acompanhamento.
Resultados	Resultados de imagem ou laboratoriais, com parâmetros, unidades e faixas de referência.
Parecer médico	Registro da interpretação médica vinculada ao resultado do exame.
Receitas	Emissão, renovação e consulta de receitas e itens.
Documentos	Atestados, laudos, relatórios, encaminhamentos e declarações.
Notificações	Avisos sobre consultas, exames, resultados, pareceres, receitas e documentos.
Histórico	Linha do tempo derivada dos registros existentes; sem tabela própria.

3.3 Perfis e níveis de acesso

Role	Permissões principais
ADMINISTRADOR	Acesso global; cadastro e gerenciamento de usuários, perfis, cadastros auxiliares, consultas e exames.
ATENDENTE	Atendimento administrativo; acesso a dados do paciente e gestão de agendamentos. Não emite parecer, receita ou solicitação médica.
MEDICO	Acesso clínico; consulta histórico e resultados, solicita exames, emite pareceres, receitas e documentos médicos.
PACIENTE	Acesso às próprias informações, consultas, exames, resultados, pareceres, receitas, documentos e notificações.

4. Processo de Desenvolvimento

A equipe adotou Spec-Driven Development (SDD), onde a especificação é a fonte de verdade e o código é consequência dela. O fluxo de trabalho seguiu as etapas:

- /specify <feature> -> specs/NNN-slug/spec.md (o QUE e o PORQUÊ, sem tecnologia)
- /plan NNN-slug -> specs/NNN-slug/plan.md (o COMO: camadas, contratos, dados)
- /tasks NNN-slug -> specs/NNN-slug/tasks.md (tarefas ordenadas e paralelizáveis)
- /implement NNN-slug -> código + testes
- /analyze NNN-slug -> auditoria de consistência

4.1 Divisão da equipe

Responsável	Frente	Entregas principais
Luis	Arquitetura e segurança	Modelo ER; Clean Architecture; USUARIO, ROLE, USUARIO_ROLE; PACIENTE; MEDICO; ESPECIALIDADE; TIPO_EXAME; configuração base do Spring Security.
Thiago	Consultas, exames, receitas	CONSULTA; SOLICITACAO_EXAME; EXAME; resultados de imagem e laboratório; RECEITA; ITEM_RECEITA.
Juliana	Clínico e qualidade	PARECER_MEDICO; DOCUMENTO_MEDICO; testes automatizados/unitários; apoio em Swagger e observabilidade.
Gilmar	Infraestrutura	Docker; Docker Compose; PostgreSQL; Flyway; apoio a armazenamento/geração de arquivos.
Danilo	Mensageria e entrega	Kafka; NOTIFICACAO; consumidores/produtores de eventos; relatório final; vídeo e apoio à apresentação.

4.2 Ferramentas e práticas

- Git com estratégia de branches por feature (ex: feature/009-notificacoes-danilo).
- Pull Requests com revisão antes do merge na main.
- Commits atômicos por fase do SDD (specify, plan, tasks, implement, analyze).
- ArchUnit para verificação automática das regras de dependência entre camadas.
- Spring Modulith para verificação de fronteiras de módulo em tempo de build.
- Gate de cobertura de 80% no perfil quality do Maven.
- Docker Compose para ambiente reproduzível local com Postgres, Kafka, Prometheus e Grafana.

5. Detalhes Técnicos

5.1 Stack de tecnologias

Tecnologia	Uso
Java 21	Linguagem principal
Spring Boot 3.5	API e infraestrutura da aplicação
Spring Modulith	Monólito modular com fronteiras verificadas em build
Spring Security 6 + JWT	Autenticação stateless e autorização por perfil
Spring Data JPA + Hibernate	Persistência e ORM
PostgreSQL 16	Banco de dados relacional
Apache Kafka 3.x (KRaft)	Mensageria e eventos de domínio
Flyway	Versionamento de migrations SQL
Docker / Docker Compose	Padronização e reprodutibilidade do ambiente
JUnit 5 + Mockito	Testes unitários e de integração
ArchUnit	Verificação de regras de arquitetura em tempo de build
Springdoc OpenAPI / Swagger UI	Documentação e exploração da API
Prometheus + Grafana	Observabilidade: métricas e dashboards
Jakarta Bean Validation	Validação declarativa de entradas

5.2 Arquitetura - Monólito Modular com Clean Architecture

A solução utiliza Spring Modulith para organizar o sistema como um monólito modular onde cada módulo de negócio é uma 'microaplicação' com Clean Architecture própria. As fronteiras de módulo são verificadas em tempo de build, o que impede que um módulo acesse os internos de outro.

```
presentation -> application -> domain <- infrastructure
```

Regras da arquitetura (verificadas automaticamente pelo ArchUnit):

- domain é Java puro: sem Spring, sem JPA, sem Jackson.
- Entidade de domínio NÃO é entidade JPA.
- Repositório é interface no domain, implementado em infrastructure.
- Um módulo só expõe o pacote api. O restante é interno.
- Nenhuma FK entre tabelas de módulos diferentes - referência via UUID validado por porta.
- O módulo notificacoes não é importado por ninguém; ele só escuta eventos.

5.3 Segurança e controle de acesso

A autenticação usa JWT stateless com Spring Security. A autorização é aplicada no caso de uso, não apenas no controller, garantindo que nenhuma lógica de negócio possa ser chamada sem as permissões corretas.

- JWT stateless; autorização com @PreAuthorize no caso de uso.
- Paciente só acessa o que é dele. O patientId vem do token, nunca da requisição.
- Recurso de terceiro responde 404, não 403 (evita enumeração de recursos).

- Nunca logado: CPF, cartão SUS, resultado, laudo, diagnóstico ou medicamento.
- Segredos apenas por variável de ambiente; nunca no código ou no repositório.

Exemplo de proteção de operação clínica:

```
@PreAuthorize("hasRole('MEDICO')")
public ParecerResponse emitirParecer(EmitirParecerCommand cmd) { ... }
```

5.4 Modelo de dados

A modelagem foi organizada por responsabilidade de domínio. Cada entidade tem papel bem definido e as referências entre módulos são feitas por UUID, sem FK direta.

Domínio	Entidades principais
Segurança	USUARIO, ROLE, USUARIO_ROLE
Pessoas	PACIENTE, MEDICO, ESPECIALIDADE, UNIDADE_SAUDE
Atendimento	CONSULTA
Exames	TIPO_EXAME, SOLICITACAO_EXAME, EXAME
Resultados	RESULTADO_EXAME, RESULTADO_IMAGEM, RESULTADO_LABORATORIAL, ITEM_RESULTADO_LABORATORIAL
Análise clínica	PARECER_MEDICO
Receitas	RECEITA, ITEM_RECEITA
Documentos	DOCUMENTO_MEDICO
Notificações	NOTIFICACAO

5.5 Fluxo de exames e resultados

O módulo de exames separa claramente a solicitação médica, a realização, o resultado técnico e o parecer clínico. Essa separação evita mistura de dados de naturezas diferentes e permite auditoria e histórico confiáveis.

1. O médico cria uma SOLICITACAO_EXAME vinculada ao paciente, à consulta e ao TIPO_EXAME.
2. O EXAME representa a execução/agendamento daquela solicitação em uma UNIDADE_SAUDE.
3. Após a realização, é criado um RESULTADO_EXAME (imagem ou laboratorial).
4. O médico analisa o resultado e registra um PARECER_MEDICO independente.
5. A partir do atendimento, o médico pode emitir RECEITA e DOCUMENTO_MEDICO.

Resultado laboratorial - exemplo de parâmetros:

Parâmetro	Resultado	Referência	Status
Hemoglobina	14,2 g/dL	13,0 a 17,0	NORMAL
Glicemia	135 mg/dL	70 a 99	ACIMA_REFERENCIA
HIV	Não reagente	Não reagente	NORMAL

5.6 Mensageria e notificações (Kafka)

Cada caso de uso clínico publica um evento de domínio dentro da própria transação. O Spring Modulith grava a publicação na tabela event_publication e, após o commit, entrega o evento ao módulo notificacoes, que cria notificações por destinatário. Com MESSAGING_ENABLED=true, o

evento é externalizado ao Kafka no padrão sus.<modulo>.<evento>.v1.

Evento	Disparado quando
ConsultaAgendadaEvent	Nova consulta criada.
ConsultaRemarcadaEvent	Alteração de data/hora da consulta.
ConsultaCanceladaEvent	Cancelamento da consulta.
ExameSolicitadoEvent	Médico solicita um exame.
ExameAgendadoEvent	Exame agendado em unidade de saúde.
ResultadoExameDisponivelEvent	Resultado técnico disponibilizado.
ParecerMedicoCriadoEvent	Médico conclui análise e registra parecer.
ReceitaEmitidaEvent	Nova receita disponibilizada.
ReceitaRenovadaEvent	Renovação de receita existente.
DocumentoMedicoEmitidoEvent	Novo documento médico disponível.

6. Diagramas de Arquitetura C4

Os diagramas C4 documentam a arquitetura em quatro níveis de detalhe progressivo, desde a visão de contexto até a estrutura interna de um módulo. Todos foram gerados automaticamente a partir de definições Mermaid e estão disponíveis como arquivos PDF individuais na pasta docs/ do repositório.

Nível 1 - Contexto do Sistema

Mostra o sistema como uma caixa preta e seus relacionamentos com os quatro tipos de usuários: Paciente, Médico, Atendente e Administrador.

Arquivo: docs/C4-Nivel1-Contexto.pdf

Nível 2 - Containeres

Detalha os cinco contêineres da plataforma e seus relacionamentos:

Container	Tecnologia	Responsabilidade
sus-api	Java 21 / Spring Boot 3.5	Monólito modular. Implementa todos os módulos de negócio. Porta 8080.
sus-postgres	PostgreSQL 16	Banco de dados relacional. Schema único com prefixos por módulo.
sus-kafka	Apache Kafka 3.x (KRaft)	Bus de eventos de domínio. Tópico padrão: sus.<módulo>.<evento>.v1.
sus-prometheus	Prometheus 2.54	Coleta métricas via /actuator/prometheus. Porta 9090.
sus-grafana	Grafana 11.2	Dashboard de observabilidade. Porta 3000.

Arquivo: docs/C4-Nivel2-Containers.pdf

Nível 3 - Componentes (Modulos Spring Modulith)

Detalha os 11 módulos dentro da sus-api e o fluxo de eventos Kafka entre eles. O módulo notificacoes é consumidor puro - nenhum outro módulo o importa. O módulo historico lê as portas api dos demais módulos em tempo de requisição, sem tabela própria de histórico.

Arquivo: docs/C4-Nivel3-Componentes.pdf

Nível 4 - Código (Clean Architecture interna)

Mostra as camadas Clean Architecture dentro do módulo consultas como exemplo representativo. Demonstra a separação entre presentation (controller), application (casos de uso), domain (entidades e repositórios, Java puro) e infrastructure (JPA, Spring).

Arquivo: docs/C4-Nivel4-Codigo.pdf

7. Escopo Prioritario e Evolucoes

7.1 Prioridades do MVP

- Cadastro, autenticação e controle por roles.
- Cadastro de pacientes, médicos, especialidades e tipos de exame.
- Agendamento e gestão de consultas.
- Solicitação e agendamento de exames.
- Registro de resultado de imagem e laboratorial.
- Parecer médico sobre o resultado.
- Emissão de receita e documentos médicos.
- Notificações orientadas a eventos.
- Histórico básico do paciente derivado dos registros existentes (sem tabela própria).

7.2 Critérios de sucesso do MVP

- Autenticação funcionando com autorização por roles.
- Fluxo demonstrável de consulta, solicitação de exame e resultado.
- Suporte a resultado de imagem e resultado laboratorial estruturado.
- Parecer médico, receita e documento vinculados corretamente ao paciente.
- Ambiente reproduzível com Docker e banco versionado por Flyway.
- Notificações desacopladas por eventos quando a mensageria estiver habilitada.

Resultado esperado do Hackathon

Entregar uma API funcional e demonstrável em que usuários autenticados atuem conforme suas roles, pacientes possam acompanhar seus atendimentos e exames, e médicos possam analisar resultados, registrar pareceres e emitir receitas e documentos.

8. Links Uteis

Acesso rápido ao repositório de código e às gravações do hackathon.

Repositório GitHub

<https://github.com/lh-borges/Fiap-Hackton-SistemSaude-Grupo-32>

Vídeo - Pitch de Apresentação

<https://www.youtube.com/watch?v=dUrBcF15nv8>

Vídeo - Demonstração do MVP

<https://youtu.be/WIOXiWBTkg0>

Documentação adicional (pasta docs/)

C4-Nivel1-Contexto.pdf	Diagrama C4 Nível 1 - Contexto do sistema
C4-Nivel2-Containers.pdf	Diagrama C4 Nível 2 - Contêineres
C4-Nivel3-Componentes.pdf	Diagrama C4 Nível 3 - Módulos e eventos
C4-Nivel4-Codigo.pdf	Diagrama C4 Nível 4 - Clean Architecture interna
mensageria-e-notificacoes.md	Guia de demonstração da mensageria e notificações

9. Aprendizados e Proximos Passos

9.1 Principais aprendizados da equipe

- Spring Modulith impõe fronteiras de módulo em tempo de build, tornando a arquitetura modular verificável e não apenas documentada. Isso exige disciplina inicial, mas elimina revisões manuais de dependência no code review.
- A separação entre entidade de domínio e entidade JPA exige mais código, mas permite evoluir cada camada de forma completamente independente, sem vazar detalhes de infraestrutura para o domínio.
- Eventos de domínio internos (Spring ApplicationEvent) e externos (Kafka) podem coexistir com um flag de feature (MESSAGING_ENABLED), o que simplificou muito os testes locais sem broker Kafka.
- ArchUnit automatiza a verificação da regra de dependência entre camadas, garantindo que nenhum desenvolvedor quebre acidentalmente a Clean Architecture.
- O gate de cobertura de 80% no perfil quality força a cobertura a crescer junto com o código, e não apenas como uma tarefa separada no final do projeto.
- A modelagem de histórico sem tabela própria (derivado dos registros existentes) simplificou o banco de dados e eliminou o risco de inconsistência entre o histórico e os dados reais.

9.2 Proximos passos e evoluções futuras

- Telemedicina e integração com sistemas oficiais de saúde (RNDS).

- Assinatura digital de receitas e documentos (ICP-Brasil).
- Suporte a DICOM para exames de imagem.
- Integração com laboratórios e serviços externos via APIs.
- Notificações push, e-mail e canais externos (WhatsApp, SMS).
- Auditoria detalhada de alterações e trilha completa de acesso.
- Dashboard administrativo e aplicativo mobile para pacientes.
- Extração progressiva dos módulos como microsserviços independentes.

10. Conclusão

A Plataforma Digital de Atendimento SUS entrega uma base consistente para digitalizar partes importantes do atendimento, mantendo separação clara entre identidade, atendimento, resultado técnico, análise médica e documentos clínicos.

O modelo entidade-relacionamento sustenta o fluxo do MVP e mantém espaço para evoluções futuras sem exigir reestruturação completa do domínio. A combinação de Spring Security, PostgreSQL, JPA/Hibernate, Kafka, Docker, Flyway e testes automatizados fornece uma base técnica adequada, enquanto a Clean Architecture ajuda a organizar responsabilidades e reduzir o acoplamento entre regras de negócio e infraestrutura.

Com 422 testes passando e cobertura de 80%+, o projeto demonstra que qualidade e agilidade podem coexistir quando a arquitetura é bem definida desde o início.

Equipe Grupo 32 - Hackathon FIAP

Luis . Thiago . Juliana . Gilmar . Danilo